endava

Gang of Four:
Creational Patterns.

Agenda

01 Introduction

02 Why Creational Patterns Exist

03 Singleton

04 Factory Method

05 Builder

What

What is a Design Pattern ?

A design pattern, is a reusable solution to a common problem in software design. They are **not code**, but templates or guidelines for solving recurring problems.

Benefits

Some but not limited to, are:

- Improve code maintainability.
- Encourage best practices.
- Provide a shared vocabulary among developers.

Groups

According to GoF, DPs are grouped by categories

- Creational.
- Structural.
- Behavioural.

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Object creation is quite simple, right ?

The most harmless piece in a block of code

For most programming languages, creating objects is the simplest instruction.

```
34  User newUser = new User();
```

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Additional problems

Tight Coupling

The code becomes tied to specific implementations.

```
Payment payment = new PaypalPayment();
```

Complex Object

Some objects require many parameters.

```
User user = new User(
                "John",
                "Doe",
                "jdoe@aol.com",
                32,
                "Sales",
                true);
```

Multiple Objects Family

Related components must be created consistently.

```
Connection connection = new PostgresConnection();
Query query = new MySQLQuery();
Transaction tx = new OracleTransaction();
```

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

When properly applied, creational patterns solve:

- Object creation logic.
- Decoupling instantiation.
- Managing complex construction.
- Reusing existing objects.

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Singleton: Ensure a class has only one instance and provide a global access point.

Pros

- Controlled access to instance.
- Saves resources.
- Global access.

Cons

- Hidden dependencies.
- Harder testing.
- Can become a global state.

Use Cases

- Configuration managers.
- Logging systems.
- Cache.
- Connection pools.

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Factory Method: Define an interface for creating objects but let subclasses decide which class to instantiate.

Pros

- Decouples object creation.
- Centralized creation logic.
- Easy to extend.

Cons

- Additional abstraction.
- Increase complexity.
- Subclasses proliferation.

Use Cases

- Selecting implementations at runtime.
- Frameworks that allow extension points.
- Parsers, serializer, transformers or format handler.

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Builder: Separate the construction of a complex object from its representation.

Pros

- Readable object construction.
- Handles optional parameters gracefully.
- Supports immutable objects and validation during construction.

Cons

- Extra boilerplate code for the builder class.
- Overkill for simple objects with few parameters.
- Two-step construction process (builder and then build()).

Use Cases

- Configuration objects.
- Complex domain models with optional fields.
- API request body or DTO construction.

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0